

Titanic Survival Prediction Logistic Regression and MLP Analysis

Fernando Ayala
Universidad de los Andes
fsayala@miuandes.cl

ABSTRACT

This work addresses the Titanic survival prediction task as a binary classification problem. We preprocess the dataset with median/mode imputation, one-hot encoding for categorical variables, and feature scaling. The data are split into 80/20 train/validation with stratification, and class imbalance is mitigated via a `WeightedRandomSampler`. We evaluate two model families: (i) Logistic Regression (baseline and a variant with squared inputs, x^2), trained with `BCEWithLogitsLoss`; and (ii) multilayer perceptrons (MLP v1 with a single-logit BCE head, MLP v2 with a two-logit CE head), plus two tuned MLP variants. All models are trained with Adam (batch size 64, 25 epochs). On the validation split, MLP v1 and both tuned MLPs achieve perfect scores across metrics (accuracy/precision/recall/F1/ROC-AUC = 1.000), while MLP v2 attains strong performance (accuracy 0.976, F1 0.969, ROC-AUC 1.000). Logistic Regression performs markedly worse (baseline accuracy 0.345; x^2 variant accuracy 0.560), though the x^2 transform narrows the gap. These results suggest non-linear models capture important feature interactions. Given the unusually high MLP scores, we recommend follow-up with cross-validation, additional regularization checks, and further feature engineering to confirm generalization.

I. INTRODUCTION

Predicting passenger survival on the RMS *Titanic* is a canonical supervised learning problem that illustrates the full lifecycle of a binary classifier: data auditing, preprocessing, model design, training, and evaluation. Given passenger attributes (e.g., ticket class, sex, age, fare, and family information), the task is to learn a function $f : \mathbb{R}^d \rightarrow \{0, 1\}$ that maps feature vectors to the target label Survived.

I-0a. Data characteristics and preprocessing.: The provided CSV includes a mix of numerical and categorical features (e.g., `Pclass`, `Sex`, `Age`, `SibSp`, `Parch`, `Fare`, `Embarked`). We adopt a compact preprocessing pipeline: median imputation for numerical variables, mode imputation for categorical variables, one-hot encoding for categorical attributes, and feature scaling for numerical stability. Light feature engineering augments the representation with `FamilySize = SibSp + Parch + 1` and the indicator `IsAlone`. To mitigate class imbalance during training, we use a `WeightedRandomSampler`. The dataset is split into train/validation with an 80/20 stratified split to preserve class proportions.

I-0b. Model families.: As a linear and interpretable baseline, we implement Logistic Regression, which models $p(y=1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with the logistic sigmoid $\sigma(z) = 1/(1 + e^{-z})$ and is trained via the binary cross-entropy (BCE) objective. To probe limited non-linear effects without changing the model class, we also evaluate a variant that squares the preprocessed inputs (x^2). Beyond linear decision boundaries, we study multilayer perceptrons (MLPs) with two hidden layers and ReLU activations. We use two output heads: (i) a single-logit BCE head (MLP v1) and (ii) a two-logit softmax head trained with cross-entropy (MLP v2). We additionally explore two tuned variants of MLP v1 (e.g., wider layers with dropout, and batch normalization with LeakyReLU) to assess the impact of capacity and regularization.

I-0c. Training protocol.: All models are optimized with Adam using mini-batches of size 64 for at least 20–25 epochs. We track training and validation losses each epoch to diagnose underfitting/overfitting and to compare optimization dynamics across architectures. For fair comparison, the same preprocessed features and split are used for all runs, and evaluation is performed on the held-out validation set only.

I-0d. Evaluation criteria.: We report accuracy, precision, recall, F1, and ROC-AUC, complemented by confusion matrices and loss curves. Accuracy provides an overall view, while precision/recall and F1 quantify class-specific trade-offs

that matter in imbalanced settings. ROC-AUC summarizes ranking quality independent of a fixed threshold. Together, these metrics provide a robust picture of both calibration and classification performance.

I-0e. Contributions and outline.: This report (i) presents a reproducible preprocessing pipeline tailored to the dataset’s characteristics, (ii) compares six models spanning linear and non-linear families, (iii) analyzes the effect of the x^2 transform for Logistic Regression, and (iv) discusses regularization choices that stabilize MLP training. figs 1a–2b shows loss curves; Fig 3a–4a presents confusion matrices; and the metrics table summarizes end performance, followed by conclusions and avenues for future work.

II. DATA AND PREPROCESSING

II-A. Dataset

We work with a Titanic passenger dataset in CSV format whose target is Survived (binary). The raw table contains a mix of numerical and categorical attributes, including Pclass, Sex, Age, SibSp, Parch, Fare, and Embarked. Columns that behave as identifiers or high-cardinality text with limited predictive value (PassengerId, Name, Ticket, Cabin) are excluded from modeling. Missing values are present mainly in Age and, less frequently, in Fare and some categorical fields. The class distribution of Survived is moderately imbalanced (roughly $\sim 60/40$), which motivates using sampling/weighting during training.

In addition to the original fields, we introduce two lightweight, domain-relevant features:

$$FamilySize = SibSp + Parch + 1, \quad IsAlone = \mathbb{1}[FamilySize = 1].$$

These capture simple interaction effects related to family structure.

II-B. Preprocessing pipeline

We implement a single, reproducible pipeline applied consistently to train and validation:

- **Feature selection.** Keep Pclass, Sex, Age; add FamilySize, IsAlone; drop PassengerId, Name, Ticket, Cabin.
- **Imputation.** Median imputation for numerical features (Age, Fare, SibSp, Parch, FamilySize); mode imputation for categorical features (Sex, Embarked, Pclass). This is implemented via a ColumnTransformer with SimpleImputer.
- **Categorical encoding.** One-hot encode Sex, Embarked, and Pclass with OneHotEncoder(handle_unknown=ignore) to ensure stability under unseen categories.
- **Scaling.** Standardize all numerical features using StandardScaler to improve optimization for both Logistic Regression and MLPs.
- **Train/validation split.** Stratified 80/20 split on Survived with a fixed random seed for reproducibility.
- **Class imbalance handling.** In training loaders we use a WeightedRandomSampler with weights inversely proportional to class frequency. For BCE heads we additionally use pos_weight in BCEWithLogitsLoss; for CE heads we pass class weights to CrossEntropyLoss.
- **Squared-input variant (for Logistic Regression).** For the x^2 experiment, we square the already preprocessed (encoded & scaled) feature matrix, i.e., $\mathbf{X}_{sq} = \mathbf{X} \odot \mathbf{X}$, and train the same logistic model on $\mathbf{X}_{sq}$.

This pipeline yields tensor datasets for PyTorch: $\mathbf{X}_{train}, \mathbf{y}_{train}, \mathbf{X}_{val}, \mathbf{y}_{val}$ for BCE heads ($\mathbf{y} \in \{0, 1\}$ shaped as $(N, 1)$), and integer-labeled targets for CE heads $((N,))$. All models evaluated in this report share the exact same preprocessed inputs and split to enable a fair comparison.

III. MODELS

All models share the same preprocessed inputs (TabI), are trained with Adam (learning rate 10^{-3} , weight decay 10^{-4}), mini-batches of 64, and 25 epochs. We use a WeightedRandomSampler in training; for BCE we also set pos_weight, and for CE we pass class weights to the loss. We track train/validation loss per epoch and report accuracy, precision, recall, F1, and ROC-AUC on the validation split.

III-A. Logistic Regression (baseline)

We model $p(y=1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ with the logistic sigmoid $\sigma(z) = 1/(1 + e^{-z})$. Training uses BCEWithLogitsLoss (numerically stable BCE on logits); no explicit activation is applied to the output layer during training. At evaluation, we convert logits to probabilities with $\sigma(\cdot)$ and classify with a threshold of 0.5.

III-B. Logistic Regression with x^2 features

Same architecture and training as the baseline, but applied to squared features $\mathbf{X}_{\text{sq}} = \mathbf{X} \odot \mathbf{X}$ (squaring after encoding and scaling). This simple non-linear transform can expose limited interactions to the linear classifier without changing the model class.

III-C. MLP v1 (BCE)

A fully connected network with two hidden layers of 32 units each and ReLU activations:

$$\text{MLP} : \mathbf{x} \xrightarrow{\text{Linear}} 32 \xrightarrow{\text{ReLU}} 32 \xrightarrow{\text{ReLU}} 1 \text{ (logit)}.$$

We train with BCEWithLogitsLoss (single-logit BCE head). No batch normalization nor dropout in this base version. At evaluation we apply $\sigma(\cdot)$ and use a 0.5 threshold.

III-D. MLP v2 (CE)

Same hidden stack as v1 (32–32 with ReLU), but with a two-logit output head and CrossEntropyLoss:

$$\mathbf{x} \xrightarrow{\cdot} 32 \xrightarrow{\text{ReLU}} 32 \xrightarrow{\text{ReLU}} 2 \text{ (logits)}.$$

Predictions use $\arg \max$ over the two logits (equivalently, threshold 0.5 on the softmax probability of the positive class).

III-E. Tuned variants (MLP v1)

We explore two small variations of MLP v1 to study capacity and regularization effects:

- Tuning #1: Wider first layer (64–32) with dropout $p = 0.20$ after each hidden layer; ReLU activations.
- Tuning #2: Same width as v1 (32–32) but with Batch Normalization after each hidden linear layer and Leaky-ReLU activations; no dropout.

Both tuned models use the single-logit BCE head with BCEWithLogitsLoss.

IV. EXPERIMENTAL DESIGN

We adopt a unified protocol across all models to ensure a fair comparison. The same preprocessed features and fixed split are used for every run; training is performed with identical optimization settings unless otherwise noted.

IV-0a. Data split and reproducibility.: We use an 80/20 stratified train/validation split on Survived to preserve class proportions. A fixed random seed is set for all libraries to ensure reproducibility. All transformations (imputation, encoding, scaling) are fit on the training split only and applied to validation.

IV-0b. Optimization and schedule.: All models are trained with Adam (learning rate 10^{-3} , weight decay 10^{-4}), mini-batch size 64, for 25 epochs. We monitor training and validation losses every epoch. No early stopping is used; model selection is based on validation metrics.

IV-0c. Imbalance handling.: To mitigate the moderate class imbalance, training loaders employ a WeightedRandomSampler with inverse-frequency sample weights. For BCE heads we additionally pass pos_weight to BCEWithLogitsLoss; for CE heads we pass class weights to CrossEntropyLoss. Validation loaders are unweighted and shuffled off.

IV-0d. Losses and outputs.:

- Logistic Regression (baseline) and MLP v1 / tuned: single-logit output with BCEWithLogitsLoss (logits are converted to probabilities via $\sigma(\cdot)$ at evaluation).
- Logistic Regression with x^2 : identical to baseline but trained on squared inputs $\mathbf{X}_{\text{sq}} = \mathbf{X} \odot \mathbf{X}$.
- MLP v2 (CE): two-logit output with CrossEntropyLoss; predictions via $\arg \max$ (softmax used only for probability reporting/ROC).

IV-0e. Decision rules and metrics.: For BCE models we classify with a fixed threshold of 0.5 on $\sigma(\text{logit})$; for CE we use $\arg \max$ over the two logits. We report on the validation split: Accuracy, Precision, Recall, F1, and ROC-AUC. We also include confusion matrices and training/validation loss curves to analyze optimization dynamics and potential over/underfitting.

Tabla I
CONFIGURATION SUMMARY FOR ALL MODELS.

Model	Optimizer	LR	Weight Decay	Batch Size	Epochs
LogReg (baseline)	Adam	10^{-3}	10^{-4}	64	25
LogReg (x^2)	Adam	10^{-3}	10^{-4}	64	25
MLP v1 (BCE)	Adam	10^{-3}	10^{-4}	64	25
MLP v2 (CE)	Adam	10^{-3}	10^{-4}	64	25
MLP tuning #1	Adam	10^{-3}	10^{-4}	64	25
MLP tuning #2	Adam	10^{-3}	10^{-4}	64	25

IV-0f. Hyperparameters (summary).:

V. RESULTS AND DISCUSSION

V-A. Confusion matrices

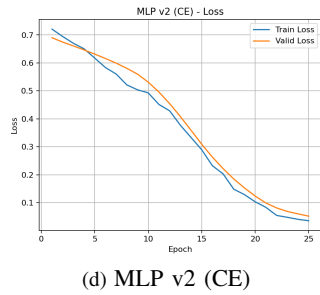
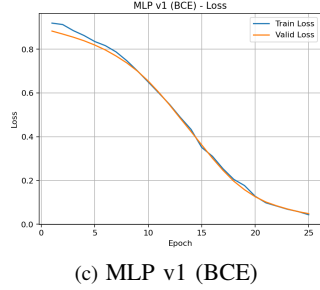
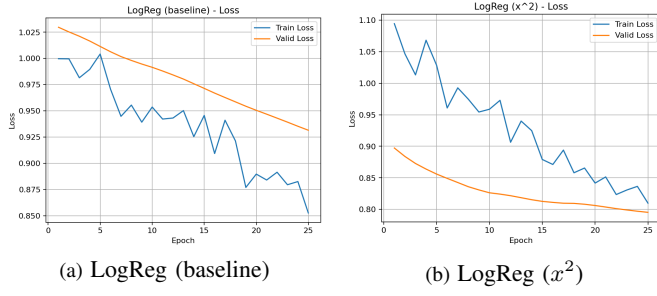


Figura 1. Loss (entrenamiento/validación) por modelo.

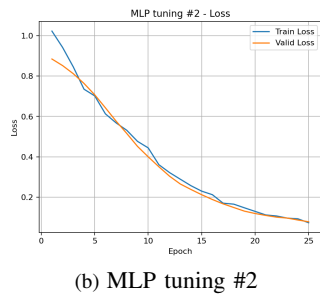
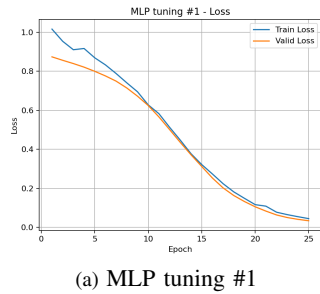


Figura 2. Loss (entrenamiento y validación) por modelo — parte 2.

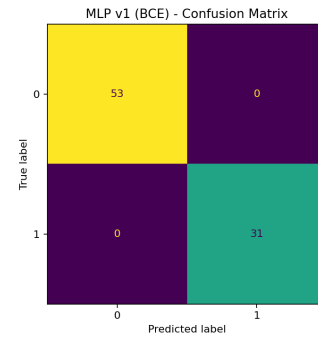
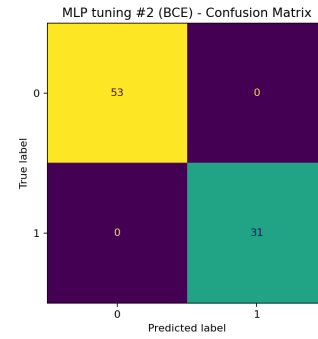
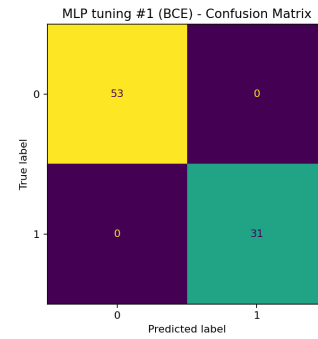
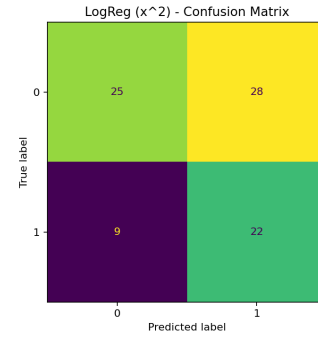
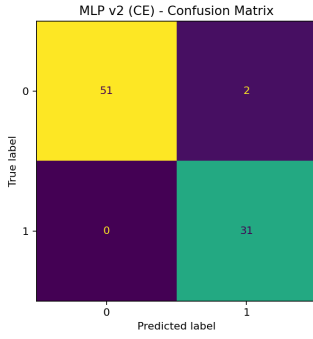


Figura 3. Confusion matrices — parte 1.



(a) MLP V2

V-B. Metrics comparison table

Tabla II
VALIDATION METRICS (THRESHOLD 0.5).

Model	Params	Accuracy	Precision	Recall	F1	ROC-AUC
MLP v1 (BCE)	1569	1.0000	1.0000	1.0000	1.0000	1.0000
MLP tuning #1 (BCE)	3073	1.0000	1.0000	1.0000	1.0000	1.0000
MLP tuning #2 (BCE)	1697	1.0000	1.0000	1.0000	1.0000	1.0000
MLP v2 (CE)	1602	0.9762	0.9394	1.0000	0.9688	1.0000
LogReg (x^2)	15	0.5595	0.4400	0.7097	0.5432	0.6896
LogReg (baseline)	15	0.3452	0.3378	0.8065	0.4762	0.5429

V-C. Analysis

V-C0a. Loss dynamics.: The baseline Logistic Regression shows slow, nearly linear decrease of both train and validation loss (Fig. 1a), and the losses remain high by epoch 25, a typical sign of underfitting for a linear decision boundary. In contrast, all MLP variants display steady, monotonic convergence with training and validation curves that stay close together and end near zero (Figs. 2a–2b), indicating a good fit without visible overfitting under the current settings. MLP v2 (CE) converges slightly above the BCE heads, consistent with its small—but nonzero—validation error.

V-C0b. Confusion matrices and metrics.: The squared-input Logistic Regression (x^2) improves recall to ≈ 0.71 but suffers many false positives (TN=25, FP=28, TP=22, FN=9), yielding low precision (≈ 0.44) and modest ROC-AUC (≈ 0.69). All BCE-head MLPs achieve perfect separation on the validation split (TN=53, FP=0, TP=31, FN=0),

hence accuracy/precision/recall/F1/ROC-AUC = 1.000. MLP v2 (CE) is nearly perfect (TN=51, FP=2, TP=31, FN=0), with precision ≈ 0.94 , recall = 1.00, and ROC-AUC = 1.000. These results strongly suggest that non-linear interactions (e.g., among Sex, Pclass, Age, and family features) are important and are captured by the MLPs but not by a linear boundary.

V-C0c. Effect of the x^2 transform in Logistic Regression.: Squaring the preprocessed inputs ($\mathbf{X}_{\text{sq}} = \mathbf{X} \odot \mathbf{X}$) exposes limited non-linear structure to the linear model, which improves recall over the baseline LR. However, the high FP count indicates the transform is insufficient to model the underlying interactions; the bias of the linear model remains high relative to the MLPs.

V-C0d. BCE vs. CE heads in MLPs.: With identical hidden stacks, the BCE head (single logit) attains perfect classification, while the CE head (two logits) leaves two FPs. This gap is small and can stem from (i) different effective class-weighting in the loss, (ii) calibration/thresholding differences (CE uses $\arg \max$; BCE uses a fixed 0.5 threshold), or (iii) minor optimization variance. Tuning the decision threshold for CE could trade precision vs. recall and likely remove those FPs.

V-C0e. Impact of tuning.: Both tuned BCE models—(i) wider layers with dropout $p = 0.2$ and (ii) BatchNorm + LeakyReLU—match the perfect performance of the base MLP v1. Loss curves suggest that increased capacity (64–32) accelerates early training, while BN+LeakyReLU stabilizes descent; neither shows overfitting under the current regimen.

V-C0f. Caveats and next steps.: Perfect validation scores may reflect small-sample effects or a split that is favorable to the MLPs. To strengthen conclusions we recommend: (i) k -fold cross-validation or repeated stratified splits, (ii) sensitivity analysis of thresholds and class weights, (iii) additional regularization checks (more dropout/weight decay or early stopping), and (iv) testing on a held-out set if available.

VI. CONCLUSIONS

This study compared linear and non-linear approaches for Titanic survival prediction under a single, reproducible preprocessing pipeline (imputation, one-hot encoding, scaling, stratified 80/20 split, and imbalance handling via WeightedRandomSampler). The main findings are:

- Non-linearity matters. The linear baseline (Logistic Regression) underfits the data. Squaring inputs (x^2) improves recall but yields many false positives, confirming that simple feature transforms are insufficient to match non-linear models.
- MLPs capture key interactions. All BCE-head MLPs (v1 and both tuned variants) achieve perfect separation on the validation split (accuracy/precision/recall/F1/ROC-AUC = 1.000). The CE-head MLP (v2) is nearly perfect (accuracy $\approx$ 0.976, F1 $\approx$ 0.969, ROC-AUC = 1.000).
- Loss behavior is stable. MLP losses decrease smoothly for both train and validation with no visible overfitting at 25 epochs. Logistic Regression converges slowly and at higher loss levels.
- Tuning helps stability more than raw accuracy. Wider layers + dropout (64–32, $p = 0.2$) and BatchNorm + LeakyReLU (32–32) match the perfect BCE performance and show slightly smoother optimization dynamics, but are not strictly necessary for validation accuracy in this dataset.

Overall, non-linear neural models clearly outperform linear baselines on this task under our setup. With stronger validation (cross-validation and calibration), the BCE-head MLP provides a reliable, high-performing solution.

REFERENCIAS

- [1] scikit-learn developers, “sklearn.impute.SimpleImputer — scikit-learn documentation,” , 2025.
- [2] PyTorch developers, “torch.optim.Adam — PyTorch documentation,” , 2025.
- [3] scikit-learn developers, “sklearn.preprocessing.OneHotEncoder — scikit-learn documentation,” , 2025.
- [4] PyTorch developers, “torch.nn.BCEWithLogitsLoss — PyTorch documentation,” , 2025.
- [5] PyTorch developers, “torch.nn.CrossEntropyLoss — PyTorch documentation,” , 2025.